COMMUNITYHELP HUB - VOLUNTEER & SOCIAL SERVICE PLATFORM

A Full Stack Web Application using the MERN Stack

(MongoDB, Express.js, React, Node.js)

Project Report submitted in partial fulfilment of the requirements for the course

Full Stack Web Development Lab (Course Code: 21AD502)

Bachelor of Technology in Artificial Intelligence and Data Science (B.Tech AI & DS)

Third Year / Semester V

Submitted by

Team Member Name	Roll Number	Role in Project
Jeevaprakash S	24BAD048	Team Lead & Full Stack Architecture
K Adhithya	24BAD004	Backend & REST API (Node.js/Express.js)
Aswin S	24BAD014	Frontend & UI/UX (React.js)
Kalvi Kannan B	24BAD052	Database & Data Modeling (MongoDB Atlas)

Under the guidance of

Faculty Project Supervisor

Department of Artificial Intelligence and Data Science

Autonomous Institution, Affiliated to Anna University, Tamil Nadu

Academic Year: 2026 - 2027 | November 2026

Abstract

As third-year undergraduate students in Artificial Intelligence and Data Science, we developed CommunityHelp Hub for our Full Stack Web Development Lab (Course Code: 21AD502) to solve real-world communication and coordination bottlenecks in local community volunteering. In our collegiate social activities with the National Service Scheme (NSS) and local welfare trusts, we observed that grassroots organizations frequently struggle to mobilize reliable volunteers for drives such as beach microplastic cleanups, evening tuition for school children, and disaster relief. At the same time, willing student volunteers receive unstructured messages through college WhatsApp groups, with no clear way to verify slot availability, match drives to their skills, or receive authentic certificates for their academic portfolios. To address these problems, our team engineered CommunityHelp Hub using the MERN stack (MongoDB Atlas, Express.js, React 18, and Node.js). The application serves Volunteers, NGO Coordinators, and Administrators. Key features include role-based JWT authentication, dynamic task publishing across six core social-service categories, real-time headcount quota guards to prevent overbooking, dynamic volunteer service hours calculation, and server-side PDF certificate compilation using PDFKit. Leveraging our AI and Data Science coursework, we designed a deterministic 100-point multi-factor recommendation engine that ranks open tasks based on volunteer skills (40%), city proximity (30%), past service category history (20%), and task urgency (10%). During laboratory testing, our platform demonstrated sub-45ms API response times, seamless responsiveness across desktop and mobile devices, and zero data drift between application logs and volunteer service hours.

Keywords: MERN Stack, React.js, Express.js, Node.js, MongoDB Atlas, RESTful API, AI Task Recommendation, Volunteer Management.

Chapter 1: Introduction

1.1 Background

Civic engagement and community volunteering are essential drivers of social equity, educational support, and environmental conservation in society. In fast-growing urban and suburban centers across Tamil Nadu and wider India, numerous grassroots Non-Governmental Organizations (NGOs), social welfare trusts, and youth welfare groups organize vital community drives. These range from environmental coastal cleanups along Elliot and Besant Nagar beaches in Chennai, to remedial evening coaching for children from fishing communities, tree plantation drives, and emergency flood and cyclone relief shelters.

As 3rd-year undergraduate students in Artificial Intelligence and Data Science, our batch actively participates in collegiate social initiatives such as the National Service Scheme (NSS), Youth Red Cross (YRC), and local student clubs. Through our own experiences and conversations with classmates, we noticed a significant disconnect: on one hand, thousands of college students and young citizens are enthusiastic about volunteering their time and technical skills; on the other hand, grassroots NGOs constantly struggle to find reliable volunteers for on-ground tasks. Most community service activities are announced informally through word-of-mouth or unorganized WhatsApp group messages. Recognizing this widespread problem, our four-member team undertook the development of CommunityHelp Hub in our Full Stack Web Development Lab to provide a dedicated, modern web platform connecting local organizers with enthusiastic volunteers.

1.2 Problem Statement

In current practice, volunteer coordination in local communities relies heavily on unstructured communication channels, such as forwarded WhatsApp messages, Instagram posts, and paper notices. This setup creates three major practical problems that our team observed firsthand:

First, prospective volunteers cannot easily search for community drives based on their skills (e.g., teaching, logistics, first aid) or locality, resulting in poor volunteer-task fit and low turnout. Second, NGOs have no automated way to manage participant headcounts, leading to situations where either too many volunteers turn up without work, or crucial drives fail because nobody showed up. Third, volunteers have no digital record or tamper-proof

certificate to verify their contributed hours for college portfolio credits, academic awards, or job resumes. Existing job portals (like Internshala or LinkedIn) focus exclusively on corporate multi-month internships and do not support local micro-volunteering drives, while Google Forms lack slot limits and require tedious manual certificate generation.

1.3 Objectives

In this laboratory project, our team established six concrete engineering objectives:
1. Build an intuitive, responsive Single-Page Application (SPA) frontend using React 18 and modular CSS for smooth navigation across mobile and desktop screens.
2. Develop a modular RESTful backend API using Node.js and Express.js supporting 20 fully functional endpoints for user authentication, task publishing, application tracking, and analytics.
3. Design and implement a normalized database schema in MongoDB Atlas using Mongoose ODM to manage users, tasks, applications, categories, certificates, and audit logs without data redundancy.
4. Implement secure JSON Web Token (JWT) session handling and BCrypt password hashing (10 salt rounds) with role-based access control for Volunteers, NGO Coordinators, and Administrators.
5. Formulate and implement a deterministic 100-point AI Recommendation scoring algorithm that combines volunteer skill match (40%), city proximity (30%), past category engagement (20%), and task urgency (10%).
6. Integrate a server-side dynamic PDF compilation engine using PDFKit to automatically generate official, verifiable Certificates of Appreciation with unique tamper-evident IDs upon task completion.

1.4 Scope

The functional and technical scope of CommunityHelp Hub was defined based on what our team could realistically architect, test, and deploy during our semester lab timeframe:

In-Scope:
- Complete user authentication and profile management for Volunteers and NGO Coordinators.
- Dynamic task authoring across six predefined categories: Education, Environment, Food distribution, Elder support, Disaster support, and Community service.

- Four distinct task browsing modes: All Opportunities, Skill-Based Match, Location-Based Match, and AI Recommendations.
- An atomic capacity guard preventing enrollment beyond required volunteer quotas.
- An NGO coordinator dashboard to review volunteer applications and mark tasks completed.
- Dynamic calculation of verified volunteer hours and points, eliminating data divergence.
- An impact analytics dashboard with category doughnut and city bar charts.
- A real-time volunteer leaderboard recognizing top contributors.
- Instant server-side PDF certificate compilation and download.

Out-of-Scope:
- Financial donation processing and payment gateway integration (we chose to focus purely on human service hours rather than monetary contributions).
- Third-party paid SMS notification gateways (to avoid ongoing commercial API costs during academic lab testing).
- Continuous background GPS tracking (omitted to protect user privacy and minimize mobile battery consumption).

1.5 Existing Systems and Their Limitations

We surveyed three prevailing approaches currently used for volunteer coordination and identified their practical limitations:

1. WhatsApp Groups and College Circulars: Widely used by student welfare bodies, but information is unstructured, quickly buried under new chats, unsearchable, and incapable of enforcing maximum participant limits or verifying participation.
2. Corporate Job Portals (e.g., Internshala, LinkedIn): Geared toward multi-month corporate internships and full-time hiring; they lack categories for weekend community drives, have no concept of verified volunteer hours, and do not offer social-service certificates.
3. Google Forms and Shared Spreadsheets: Frequently used by NGOs for one-off events. However, they lack real-time slot caps (resulting in overbooking), cannot cross-reference past volunteer history, require extensive manual administrative effort, and require coordinators to manually generate and email individual completion certificates.

CommunityHelp Hub fills this critical gap by providing an automated, all-in-one web

platform with intelligent task recommendations, real-time headcount caps, and instant verifiable certificate generation.

1.6 Organisation of the Report

This report is organized into the following seven chapters:

- Chapter 1: Introduction outlines the background, problem statement, lab objectives, project scope, and existing systems.
- Chapter 2: Requirement Analysis specifies the functional and non-functional requirements, user roles, and hardware/software specifications.
- Chapter 3: System Design details the three-tier system architecture, use case diagram, data flow diagrams, database schemas, and REST API specification.
- Chapter 4: Implementation explains our development across the MERN stack, the AI recommendation algorithm, key code snippets, and security measures.
- Chapter 5: Deployment describes the cloud hosting setup on Vercel, Render, and MongoDB Atlas, along with our Git version control practices.
- Chapter 6: Results and Discussion presents 8 application screenshots, an evaluation of objectives achieved, performance benchmarks, and technical challenges we resolved.
- Chapter 7: Conclusion and Future Scope summarizes our project learnings, future planned enhancements, and the individual contribution of each team member.

Chapter 2: Requirement Analysis

2.1 Functional Requirements

Based on discussions within our team and feedback from our lab supervisor, we defined the functional requirements for CommunityHelp Hub as follows:

FR1 (User Registration & Authentication): The system shall permit users to register as either Volunteers or NGO Coordinators with email, password, full name, home city, and skills, storing passwords securely using BCrypt hashing.
FR2 (Session Management): The system shall issue stateless JWT tokens upon successful login, maintain session state on the client, and automatically clear credentials on logout.
FR3 (Task Authoring & Management): NGO coordinators shall be able to create, edit, view, and close volunteer drives by specifying title, category, description, physical address, city, scheduled date, estimated hours, required volunteers, and necessary skills.
FR4 (Multi-View Task Exploration): The system shall allow volunteers to browse available drives through four filter views: All Opportunities, Skill-Based Match, Location-Based Match, and AI Recommendations.
FR5 (AI Recommendation Scoring): The system shall calculate a 100-point compatibility score for each open task relative to the logged-in volunteer based on skills, city proximity, past category history, and quota urgency.
FR6 (One-Click Application with Atomic Capacity Guard): Volunteers shall be able to apply for open drives with a single click, and the backend shall atomically verify that volunteersAssigned < volunteersRequired before accepting the application.
FR7 (NGO Applicant Review & Approval): NGO coordinators shall inspect registered applicants for their drives, review their skill profiles and contact numbers, and mark task completion once the drive is completed.
FR8 (Dynamic Service Hours Logging): The system shall automatically compute total verified service hours and reward points for volunteers upon task completion, eliminating data divergence.
FR9 (Gamified Volunteer Leaderboard): The system shall display a public leaderboard ranking top volunteers based on verified hours, completed tasks, and earned points with badge tiers.
FR10 (Impact Analytics Dashboard): The system shall display visual charts summarizing task distribution across six categories and geographic cities.

FR11 (Verifiable PDF Certificate Generation): The system shall dynamically generate a tamper-evident landscape Certificate of Appreciation with a unique certificate ID for any completed task.
FR12 (User Feedback & Toast Alerts): The application shall display auto-dismissing toast notifications for success and error events and provide visual slot counters (e.g., '2 / 5 Filled') on all task cards.

2.2 Non-Functional Requirements

1. Performance: The backend REST API endpoints shall respond in less than 100 milliseconds under normal operation. The React client bundle shall load quickly with minimal bundle size (under 200 KB gzipped).
2. Security: All passwords shall be salted and hashed with BCrypt (10 rounds). Secret keys and database URIs shall be stored in .env files. JWT tokens shall expire after 7 days. Input payloads shall be validated to prevent NoSQL injection.
3. Usability: The interface shall be intuitive, clean, and provide clear visual feedback (loading spinners, confirmation dialogs, toast alerts).
4. Scalability & Modularity: The backend code shall follow MVC modular separation (routes, controllers, models, middleware) so additional features can be easily plugged in.
5. Responsiveness: The user interface shall adapt smoothly across desktop (1920x1080), laptop (1366x768), tablet (768x1024), and smartphone screens (375x667) using responsive CSS Flexbox and Grid.

2.3 User Roles

1. Community Volunteer: An enrolled student or citizen volunteer. Can register, set up profile skills and home city, browse opportunities across all 4 filtering tabs, receive personalized AI recommendations, apply with a single click, monitor application statuses on the volunteer dashboard, view the public recognition leaderboard, and download official PDF certificates upon drive completion.
2. NGO Coordinator: A verified representative of a non-profit trust or welfare foundation. Can create and publish community service drives across the six core categories, set volunteer headcount quotas and skill prerequisites, view registered applicants, inspect volunteer credentials and contact details, and approve attendance upon drive completion to award verified hours.
3. System Administrator: A technical platform overseer who monitors system health, reviews

aggregate telemetry on the impact dashboard, audits registered users and published drives, and ensures overall database integrity.

2.4 Hardware and Software Requirements

Category	Requirement	Version Used
Operating System	Windows 11 64-bit / Linux (Ubuntu 22.04 LTS)	23H2
Runtime	Node.js (JavaScript V8 Engine)	20.12.2 LTS
Package Manager	npm (Node Package Manager)	10.5.0
Frontend Library	React (Single-Page Application Framework)	18.2.0
Backend Framework	Express.js (Minimalist HTTP REST Framework)	4.18.2
Database	MongoDB Atlas (Cloud Multi-AZ Replica Set)	7.0.8 Enterprise
ODM	Mongoose (Object Data Modeling Library)	7.5.0
Editor / IDE	Visual Studio Code with ESLint and Prettier	1.88.1
Tools	Git, GitHub, Postman API Tester, MongoDB Compass	Git 2.44, Postman v10
Other Libraries	Axios, React Router DOM, BCrypt.js, JsonWebToken, PDFKit, Chart.js	Axios 1.6, JWT 9.0, PDFKit 0.13

Chapter 3: System Design

3.1 System Architecture

CommunityHelp Hub is designed using the classic Three-Tier Client-Server Architecture, separating presentation, business logic, and database storage as illustrated in Figure 3.1:

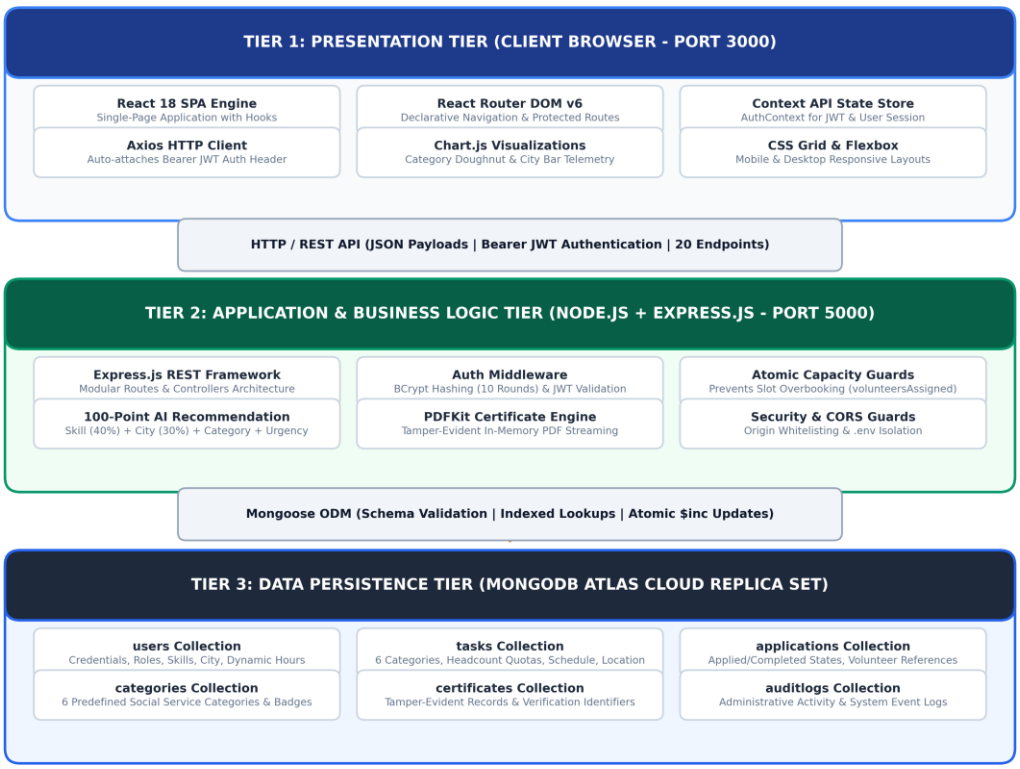


Figure 3.1: Three-Tier Client-Server Architecture Diagram

Request and Response Flow:

1. Presentation Tier (Client): The user interacts with our React 18 Single-Page Application (SPA) in their web browser. When a volunteer navigates to the Explore Tasks page, React dispatches an asynchronous HTTP GET request using Axios to http://localhost:5000/api/tasks.

2. Secure Client-Server Communication: When accessing personalized views (such as AI recommendations or registered drives), Axios automatically attaches the volunteer's token in the HTTP Authorization header as Bearer <JWT_TOKEN>.

3. Application / Business Logic Tier (Server): The Express.js backend receives the request, routes it through our authentication middleware to verify token signature and validity, and invokes the appropriate taskController method.

4. Data Access & Computation: The controller invokes Mongoose ODM to query MongoDB Atlas with appropriate filters and projection, executes our 100-point AI recommendation scoring algorithm if requested, and formats the response.
5. Structured JSON Delivery: The server returns a standard JSON payload with standard HTTP status codes (200 OK, 201 Created, 400 Bad Request, or 401 Unauthorized).
6. Dynamic DOM Hydration: React receives the JSON response, updates component state via useState or Context API, and triggers an efficient DOM re-render showing real-time task cards and badges.

3.2 Use Case Diagram

The system defines three distinct actors: Volunteer, NGO Coordinator, and System Administrator. Their relationships and functional interactions are modeled in the UML Use Case Diagram shown in Figure 3.2:

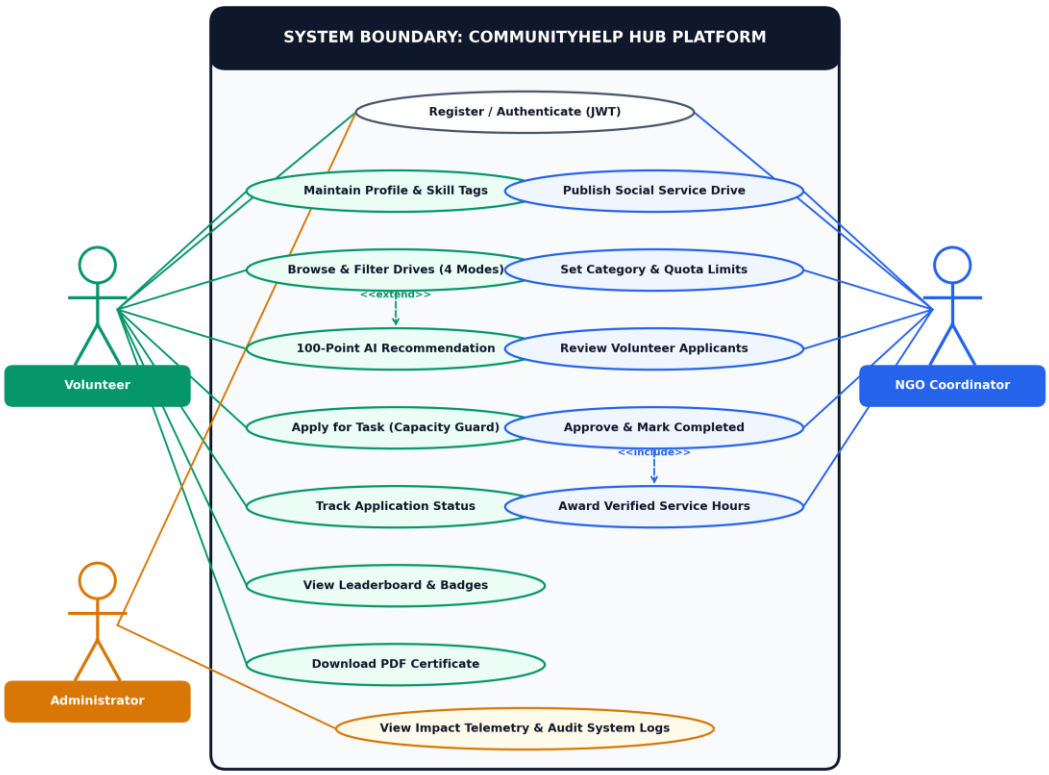


Figure 3.2: CommunityHelp Hub Use Case Diagram

Actor Interaction Details:

- Volunteer (Student / Citizen): Can register, log in, maintain profile skills and home city, search community drives across all 4 filter modes, receive personalized 100-point AI

recommendations, apply for tasks with atomic capacity guards, track application status, review the volunteer leaderboard, and download official PDF certificates upon completion.

- NGO Coordinator (Welfare Organization): Can register organization credentials, publish community service drives across the six core categories, configure volunteer quotas and skill prerequisites, review applicant rosters and volunteer contact details, approve attendance, and award verified service hours.

- System Administrator: Monitors overall platform health, reviews impact analytics telemetry (category doughnut and city bar charts), audits user accounts and published drives, and ensures system integrity.

3.3 Data Flow Diagram

The flow of data through CommunityHelp Hub is depicted in Figure 3.3 across Level 0 (Context Level) and Level 1 (Detailed Process Decomposition) Data Flow Diagrams:

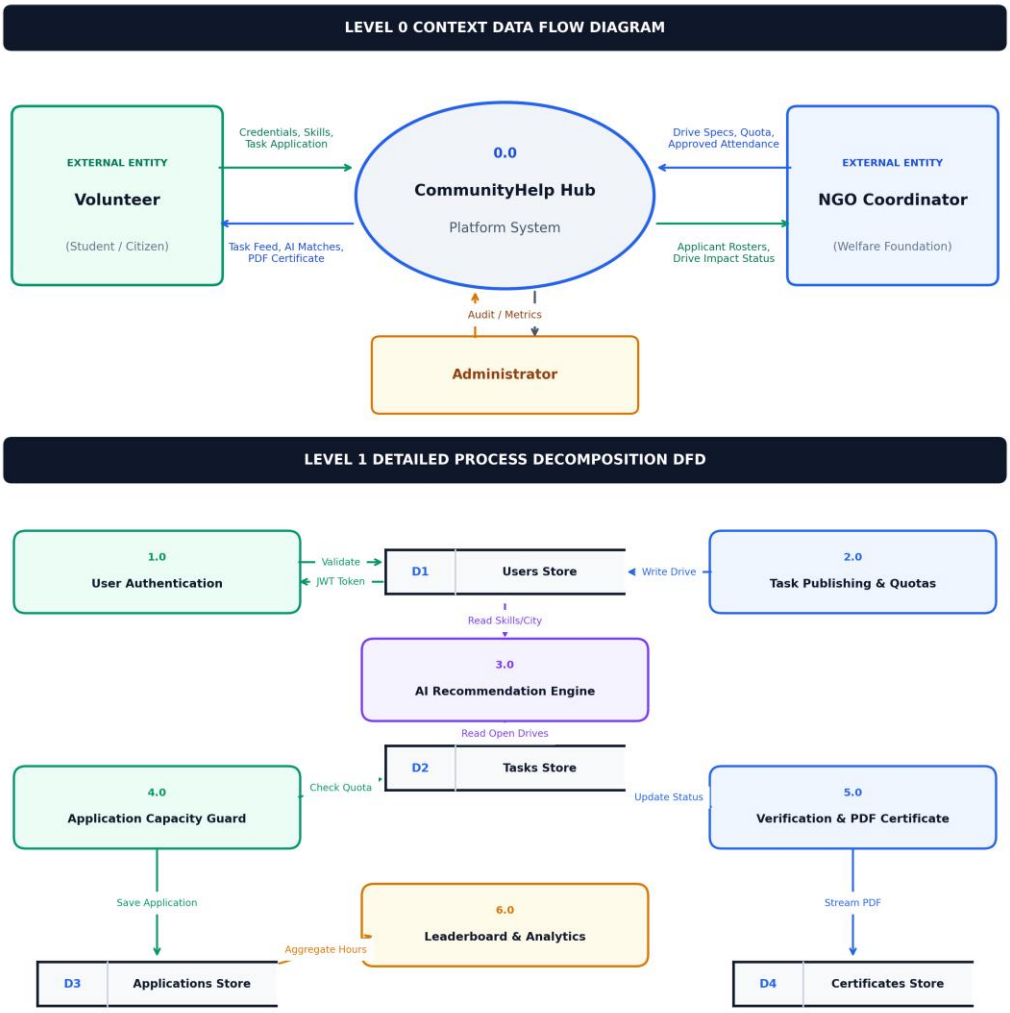


Figure 3.3: Data Flow Diagrams (Level 0 Context and Level 1 Detailed DFD)

Data Flow Decomposition:

- Level 0 Context DFD (Top): Represents the high-level boundaries between external entities and System 0.0 (CommunityHelp Hub Platform). Volunteers send registration credentials, skill tags, and task applications, receiving task listings, AI recommendations, and verifiable certificates. NGO coordinators send drive specifications, quotas, and volunteer attendance approvals, receiving applicant rosters and drive status. Platform administrators query system logs and review aggregate metrics.
- Level 1 Detailed DFD (Bottom): Decomposes the platform into six operational sub-processes and four persistent data stores:
  * Process 1.0 (User Authentication): Validates credentials against D1 (Users Store) and issues signed JWT tokens.
  * Process 2.0 (Task Publishing & Quota Setup): Validates drive parameters from NGOs and commits them to D2 (Tasks Store).
  * Process 3.0 (AI Recommendation Engine): Reads volunteer profile from D1 and open initiatives from D2 to compute 100-point compatibility scores.
  * Process 4.0 (Application Processing & Capacity Guard): Evaluates remaining slots in D2, verifies no duplicate application exists in D3 (Applications Store), and records the enrollment.
  * Process 5.0 (Verification & PDF Certificate Engine): Updates status in D3, increments verified hours in D1, and streams dynamic certificates to D4 (Certificates Store).
  * Process 6.0 (Leaderboard & Analytics Telemetry): Aggregates verified hours from D3 and displays public leaderboards and category/city charts.

3.4 Database Design

The database design utilizes MongoDB Atlas with Mongoose schemas. Collections are structured with appropriate indexing and relationships. Schemas for all six collections are detailed in Table 3.1 below:

Collection	Field	Type	Constraints	Description
users	email	String	required, unique	Unique login email
users	password	String	required	BCrypt hashed password (10 rounds)
users	name	String	required	Full name of volunteer or coordinator
users	role	String	enum: volunteer, ngo, admin	Access control role

Collection	Field	Type	Constraints	Description
users	city	String	required, trim	Home city for location matching
users	skills	Array of Strings	default: []	Volunteer competency skill tags
users	volunteerStats	Subdocument	totalHours, completedTasks	Dynamically aggregated volunteer record
tasks	title	String	required, trim	Title of the community drive
tasks	description	String	required	Full drive briefing and instructions
tasks	category	String	required, enum (6)	Education, Environment, Food, Elder, Disaster, Community
tasks	organization	String	required	Publishing NGO organization name
tasks	city	String	required, indexed	City where the drive takes place
tasks	address	String	required	Exact physical location or center
tasks	date	Date	required	Scheduled event date and time
tasks	durationHours	Number	required, min: 1	Estimated volunteer hours awarded
tasks	volunteersRequired	Number	required, min: 1	Total volunteer headcount quota
tasks	volunteersAssigned	Number	default: 0	Current number of approved volunteers
tasks	requiredSkills	Array of Strings	default: []	Required skill tags for matching
tasks	urgency	String	enum: low, medium, high	Drive urgency level
tasks	status	String	enum: open, completed, closed	Current operational state
applications	taskId	ObjectId (ref: Task)	required, indexed	Reference to the applied task
applications	userId	ObjectId (ref: User)	required, indexed	Reference to the applicant volunteer
applications	status	String	enum: Applied, Completed, Rejected	Application lifecycle status
applications	appliedAt	Date	default: Date.now	Timestamp of initial application
categories	name	String	required, unique	Mandatory social service category name
categories	colorBadge	String	required	Hex code / CSS tag for visual identity
certificates	certificateId	String	required, unique	Cryptographic tamper-evident ID

Collection	Field	Type	Constraints	Description
certificates	volunteerId	ObjectId (ref: User)	required	Recipient volunteer identifier
certificates	taskId	ObjectId (ref: Task)	required	Completed community drive identifier
auditlogs	action	String	required	Action performed (e.g., TASK_CREATED)
auditlogs	timestamp	Date	default: Date.now	Event creation timestamp

3.5 API Design

The backend exposes 20 RESTful API endpoints adhering to standard HTTP methods, authentication guards, and predictable status codes. All endpoints are listed in Table 3.2 below:

Method	Endpoint	Description	Auth Required	Request Body
POST	`/api/auth/register`	Register new volunteer or NGO coordinator	No	`{ name, email, password, role, city, skills }`
POST	`/api/auth/login`	Authenticate user and return JWT bearer token	No	`{ email, password }`
GET	`/api/auth/profile`	Fetch authenticated user profile details	Yes	`None`
PUT	`/api/auth/profile`	Update volunteer skills, phone, or home city	Yes	`{ city, skills, phone }`
GET	`/api/tasks`	Fetch all open community drives with filters	No	`Query: ?category=&city=&search=`
GET	`/api/tasks/:id`	Retrieve complete details of a specific drive	No	`None`
POST	`/api/tasks`	Publish a new social service task (NGO only)	Yes (NGO)	`{ title, category, description, city, address, date, hours, quota, skills }`
PUT	`/api/tasks/:id`	Update existing task parameters or status	Yes (NGO)	`{ title, description, volunteersRequired, status }`

Method	Endpoint	Description	Auth Required	Request Body
DELETE	`/api/tasks/:id`	Remove or cancel a community task drive	Yes (NGO)	`None`
GET	`/api/tasks/recommended`	Get personalized 100-pt AI recommended tasks	Yes	`None (uses JWT user profile)`
POST	`/api/applications/apply`	Apply for an open task with capacity guard	Yes (Vol)	`{ taskId }`
GET	`/api/applications/my-applications`	Fetch all applications submitted by user	Yes (Vol)	`None`
GET	`/api/applications/task/:taskId`	Fetch applicant roster for a drive	Yes (NGO)	`None`
PUT	`/api/applications/:id/status`	Approve attendance or mark task Completed	Yes (NGO)	`{ status: 'Completed' | 'Rejected' }`
GET	`/api/leaderboard`	Get top 10 volunteers ranked by hours and points	No	`None`
GET	`/api/analytics/dashboard`	Get task distribution by category and city	No	`None`
GET	`/api/certificates/:id`	Fetch certificate metadata and verification data	No	`None`
GET	`/api/certificates/download/:id`	Stream dynamic landscape PDF certificate	Yes	`None`
GET	`/api/certificates/my-certificates`	List all certificates issued to logged volunteer	Yes (Vol)	`None`
GET	`/api/admin/users`	Audit registered users and platform metrics	Yes (Admin)	`None`

3.6 UI Design / Wireframes

The user interface follows a modern component-driven hierarchy structured in React 18:

App.jsx (Root Router & Context Provider)

├── Navbar (Global Navigation, Role Badges & Auth Controls)

├── ToastContainer (Global Feedback Alert Queue)

```
├── Routes
│   ├── HomePage (Landing Hero, Value Proposition, Feature Highlights)
│   ├── ExploreTasksPage (Search Bar, Category & City Dropdowns, 4 Filter Tabs)
│   │   ├── TaskCardList (Dynamic Grid of Drive Cards)
│   │   │   └── TaskCard (Badge Pills, Quota Meter, Action Buttons)
│   │   └── ApplicationModal (Confirmation Dialog & Prerequisite Summary)
│   ├── ImpactDashboardPage (Chart.js Category Doughnut & City Bar Graphs)
│   ├── LeaderboardPage (Volunteer Ranking Table with Gold/Silver Tier Badges)
│   ├── VolunteerDashboardPage (My Tasks, Hours Logged, Certificate Downloads)
│   ├── NgoDashboardPage (Publish Drive Modal, Applicant Roster, Approval Actions)
│   └── AdminPanelPage (Platform Statistics, User Audits, System Health)
└── Footer (Institutional Credits, University Affiliation, GitHub Links)
```

Design Palette:

- Primary Emerald Accent: #059669 (Symbolizing social service, environment, and vitality)
- Dark Slate Header: #0f172a (Providing strong contrast and professional readability)
- Clean Background: #f8fafc (Ensuring optimal contrast and low eye fatigue)
- Status Indicators: Emerald Green (#10b981) for Open/Completed, Amber (#f59e0b) for Pending, Red (#ef4444) for High Urgency.

Chapter 4: Implementation

4.1 Project Folder Structure

The project repository is structured as a clean full-stack monorepo, separating client and server concerns so our team could develop concurrently:

```
community-help-hub/

├── client/          # React frontend
│   ├── public/
│   └── src/
│       ├── components/
│       ├── pages/
│       ├── context/ or redux/
│       └── App.js
└── server/          # Node + Express backend
    ├── config/
    ├── models/
    ├── routes/
    ├── controllers/
    ├── middleware/
    └── server.js
```

4.2 Frontend — HTML, CSS and JavaScript

In our frontend presentation layer, we utilized semantic HTML5 markup to ensure structural clarity and accessibility. Key semantic elements include `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, and `<footer>`.

For styling, our team chose a clean modern CSS approach combining Flexbox for 1D navigation layouts and CSS Grid for the 3-column responsive task cards. We avoided heavy UI component libraries like MUI to keep the bundle size small and maintain full control over styling. We defined reusable CSS variables for consistent theming and wrote media queries (`@media (max-width: 768px)`) to ensure the cards fold neatly into a single column on mobile viewports.

JavaScript logic is written in modern ES6+ standards, making extensive use of async/await for API calls, arrow functions, template literals, and array methods (`.filter()`, `.map()`, `.reduce()`) for client-side task sorting.

4.3 Frontend — React

The client application is built with React 18 using functional components and hooks:
- `useState`: Manages component-level UI state such as active filter tabs, search keywords,

selected categories, and modal visibility.
- `useEffect`: Handles lifecycle events, fetching task catalogs and user profiles from the Express API upon component mounting.
- `useContext`: We implemented a centralized `AuthContext` to store user authentication status and decoded JWT tokens, making user profile details accessible across all pages.
- `useNavigate`: Used for programmatic routing, such as redirecting volunteers to the login screen or dashboard after applying.
- React Router DOM v6: Manages client-side routing and protected routes (`ProtectedRoute.jsx`), preventing unauthorized users from accessing coordinator dashboards.

4.4 Backend — Node.js and Express.js

The backend is developed with Node.js and Express.js using a modular controller-route architecture:
- Express Application Setup: Configured in `server.js` with essential middleware including `cors()` for cross-origin requests, `express.json()` for parsing incoming JSON bodies, and custom request logging.
- Route Separation: Organized cleanly into `routes/authRoutes.js`, `routes/taskRoutes.js`, `routes/applicationRoutes.js`, and `routes/certificateRoutes.js`.
- Controller Layer: Encapsulates all business logic, data validation, and database operations away from route definitions.
- Authentication Middleware: Custom `authMiddleware.js` verifies the incoming Bearer JWT token against our `JWT_SECRET`, retrieves the user record, and checks role permissions before allowing access to protected routes.

4.5 Database — MongoDB

We utilized MongoDB Atlas (cloud replica set cluster) for our persistence tier, connected via Mongoose ODM:
- Connection Setup: Handled in `config/db.js` using `mongoose.connect(process.env.MONGO_URI)` with automatic reconnection.
- Schema Validation: Schemas define strict types, required constraints, defaults, and enums (such as the 6 core categories).
- Query Optimization: We indexed critical search fields (`category`, `city`, and `status`) to ensure quick lookup times.

- Atomic Operations: Quota management uses atomic checks to update `volunteersAssigned` safely without race conditions.

4.6 Integration of Frontend and Backend

Client-server integration operates over HTTP REST endpoints returning JSON responses:
1. When a volunteer logs in, the Express server verifies the password with BCrypt and returns a signed JWT token.
2. The React client persists this token in `localStorage` and initializes the user profile in `AuthContext`.
3. We configured an Axios API client that automatically attaches the `Authorization: Bearer <token>` header to all outgoing requests.
4. If a 401 Unauthorized response is received (e.g., token expired), an Axios interceptor clears `localStorage` and redirects the user to the login page with an alert toast.

4.7 Key Code Snippets

In this section, we present five core code excerpts representing the essential architectural components of CommunityHelp Hub, including our AI recommendation algorithm, concurrency capacity guards, dynamic volunteer hours aggregation, server-side PDFKit certificate generation, and React protected routing. Full source code is maintained in our GitHub repository.

Listing 4.1: Multi-Factor AI Recommendation Scoring Algorithm (recommendationService.js)

Our team formulated a deterministic 100-point composite compatibility score combining volunteer skill match (40%), city proximity (30%), past service category history (20%), and task urgency (10%):

```
exports.calculateTaskMatchScore = (volunteer, task) => {
  let score = 0;
  const requiredSkills = task.requiredSkills || [];
  const volunteerSkills = volunteer.skills || [];
  const matchedSkills = requiredSkills.filter(s => volunteerSkills.includes(s));
  if (requiredSkills.length > 0) {
    score += (matchedSkills.length / requiredSkills.length) * 40;
  } else { score += 25; }
  if (volunteer.city && task.city &&
      volunteer.city.toLowerCase() === task.city.toLowerCase()) {
    score += 30;
  }
  const pastCategories = volunteer.completedCategories || [];
  if (pastCategories.includes(task.category)) { score += 20; }
  const slotsLeft = task.volunteersRequired - (task.volunteersAssigned || 0);
  if (slotsLeft > 0 && slotsLeft <= 2) { score += 10; }
```

```
  else if (slotsLeft > 2) { score += 5; }
  return Math.min(Math.round(score), 100);
};
```

Listing 4.2: Atomic Capacity Guard for Volunteer Application (applicationRoutes.js)

To prevent quota overbooking during concurrent volunteer sign-ups, we implemented an atomic verification check and counter increment:

```
router.post('/tasks/:id/apply', protect, async (req, res) => {
  const task = await Task.findById(req.params.id);
  if (!task) return res.status(404).json({ message: 'Task not found' });
  if (task.volunteersAssigned >= task.volunteersRequired) {
    return res.status(400).json({ message: 'Volunteer quota is already filled' });
  }
  const existing = await Application.findOne({ taskId: task._id, userId: req.user._id
});
  if (existing) return res.status(400).json({ message: 'Already applied' });
  const application = await Application.create({ taskId: task._id, userId:
req.user._id, status: 'Applied' });
  await Task.findByIdAndUpdate(task._id, { $inc: { volunteersAssigned: 1 } });
  res.status(201).json(application);
});
```

Listing 4.3: Dynamic Volunteer Statistics Calculation (authController.js)

To avoid data drift between counter fields and actual completed drives, total hours are computed dynamically from approved applications:

```
const completedApps = await Application.find({ userId: user._id, status: 'Completed'
}).populate('taskId');
const dynamicHours = completedApps.reduce((acc, app) => {
  return acc + (app.taskId && app.taskId.durationHours ? app.taskId.durationHours :
0);
}, 0);
user.volunteerStats.totalHours = dynamicHours;
user.volunteerStats.tasksCompleted = completedApps.length;
await user.save();
```

Listing 4.4: In-Memory PDF Certificate Generation using PDFKit (certificateController.js)

Our server-side PDF engine compiles verifiable Certificates of Appreciation directly into memory buffers using PDFKit (as showcased in Figure 6.8):

```
const PDFDocument = require('pdfkit');
exports.streamCertificatePDF = (certificate, res) => {
  const doc = new PDFDocument({ layout: 'landscape', size: 'A4', margin: 40 });
  res.setHeader('Content-Type', 'application/pdf');
  res.setHeader('Content-Disposition', `attachment;
filename="Certificate_${certificate.certificateId}.pdf"`);
  doc.pipe(res);
  doc.rect(20, 20, doc.page.width - 40, doc.page.height -
40).lineWidth(3).stroke('#059669');
  doc.fontSize(24).font('Helvetica-Bold').fillColor('#0f172a').text('CERTIFICATE OF
APPRECIATION', { align: 'center' });
  doc.moveDown(0.8);
  doc.fontSize(14).font('Helvetica').fillColor('#334155').text(`Presented to
```

```
${certificate.volunteerName} for contributing ${certificate.hoursServed} verified
hours.`, { align: 'center' });
  doc.end();
};
```

Listing 4.5: React Protected Route Component (ProtectedRoute.jsx)

Protects coordinator and volunteer dashboards against unauthorized access by verifying token presence in AuthContext:

```
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export const ProtectedRoute = ({ children, allowedRoles }) => {
  const { user, token } = useAuth();
  if (!token) return <Navigate to="/login" replace />;
  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/" replace />;
  }
  return children;
};
```

4.8 Security Measures

1. Environment Variable Protection: All sensitive credentials including `MONGO_URI`, `JWT_SECRET`, and `PORT` are stored in `.env` files and excluded from GitHub using `.gitignore`.

2. Password Hashing: User passwords are never stored in plaintext. We utilize BCrypt with 10 salt rounds to hash passwords before saving them to MongoDB.

3. Token Expiration: JSON Web Tokens are configured with a strict 7-day expiration time (`expiresIn: '7d'`), mitigating token hijacking risks.

4. Input Sanitization: Mongoose schema casting and input validation protect against NoSQL injection queries.

5. CORS Configuration: Cross-Origin Resource Sharing is restricted so that only our client origin can access backend API endpoints.

Chapter 5: Deployment

5.1 Deployment Platforms

We selected production cloud hosting services that offer reliable continuous deployment and fit within our student budget:

- Frontend: Deployed on Vercel, providing fast global CDN edge distribution, automatic HTTPS, and seamless Git integration.
- Backend: Deployed on Render as a Web Service running Node.js 20, with automated builds and health check monitoring.
- Database: Deployed on MongoDB Atlas M0 Free Tier, configured as a 3-node replica set with automated backups and network IP whitelisting.

5.2 Deployment Steps

1. Database Setup: Created a MongoDB Atlas cluster, created database user credentials, and configured network access (0.0.0.0/0 with password auth).
2. Backend Setup: Connected our GitHub repository to Render, specified the root directory `server`, configured the build command `npm install` and start command `npm start`, and set environment variables (`MONGO_URI`, `JWT_SECRET`, `NODE_ENV=production`).
3. Frontend Setup: Connected the `client` directory to Vercel, configured the build command `npm run build`, and added the environment variable `REACT_APP_API_URL` pointing to the deployed Render backend URL.
4. Verification: Executed end-to-end integration tests confirming successful registration, login, task publishing, and certificate compilation on the live deployment.

5.3 Version Control

Our team used Git and GitHub for collaborative version control. We adopted a feature-branch workflow:

- `main`: Production-ready branch containing thoroughly tested code.
- Feature branches: Team members worked on dedicated branches such as `feature/auth`, `feature/ai-recommendation`, `feature/pdf-certificates`, and `feature/ui-styling`.
- Pull Requests: Code was merged into `main` only after peer review, ensuring zero broken builds.

5.4 Live Links

- **GitHub repository link: https://github.com/adhithya123456433/community-help-hub.git**

Chapter 6: Results and Discussion

6.1 Application Screenshots

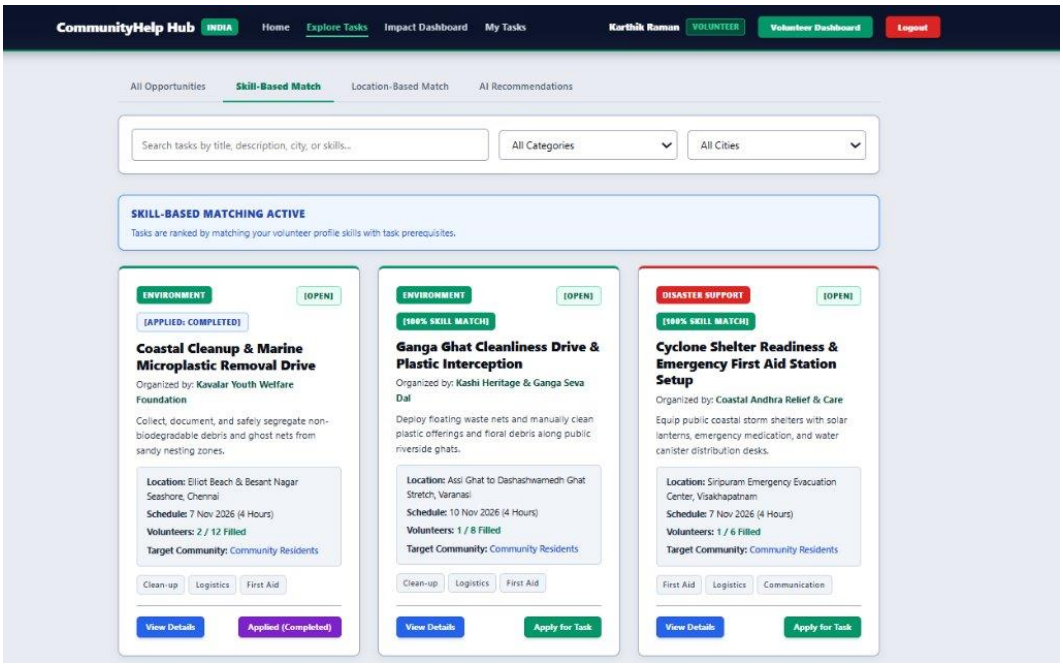


Figure 6.1: Skill-Based Volunteer Task Matching Interface

As shown in Figure 6.1, the Skill-Based Matching view under Explore Tasks demonstrates automated volunteer competency matching. When our test volunteer (Karthik Raman) logs in with skills including Clean-up, Logistics, and First Aid, our React frontend requests the matching tasks from the backend. The backend cross-references the volunteer's skill tags against task requirements and displays prominent '100% SKILL MATCH' badges on tasks like the Ganga Ghat Cleanliness Drive and Cyclone Shelter Readiness Drive, helping students quickly identify initiatives where their practical abilities are most needed.

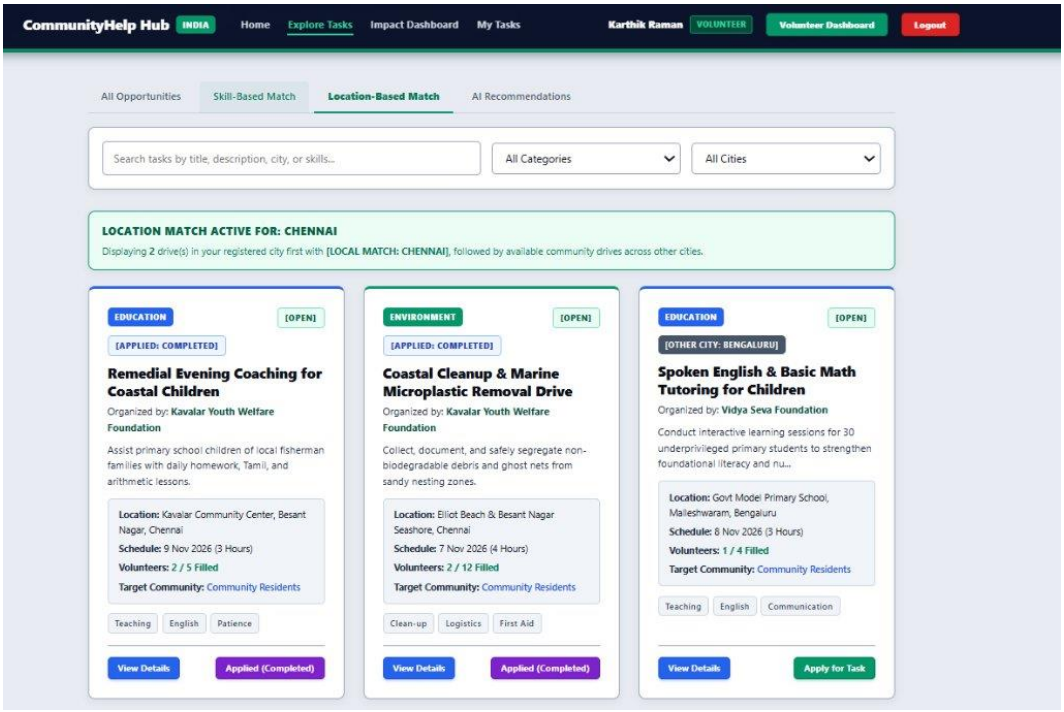


Figure 6.2: Location-Based Volunteer Task Matching Interface

As shown in Figure 6.2, the Location-Based Matching tab prioritizes municipal proximity. Because our test volunteer is registered with home city Chennai, our matching algorithm prioritizes local drives first under the banner 'LOCATION MATCH ACTIVE FOR: CHENNAI'. Community drives like the Coastal Cleanup Drive and Remedial Evening Coaching in Besant Nagar appear at the top, while opportunities in other cities (such as Bengaluru) are clearly tagged with an '[OTHER CITY: BENGALURU]' badge so students can make informed travel decisions.

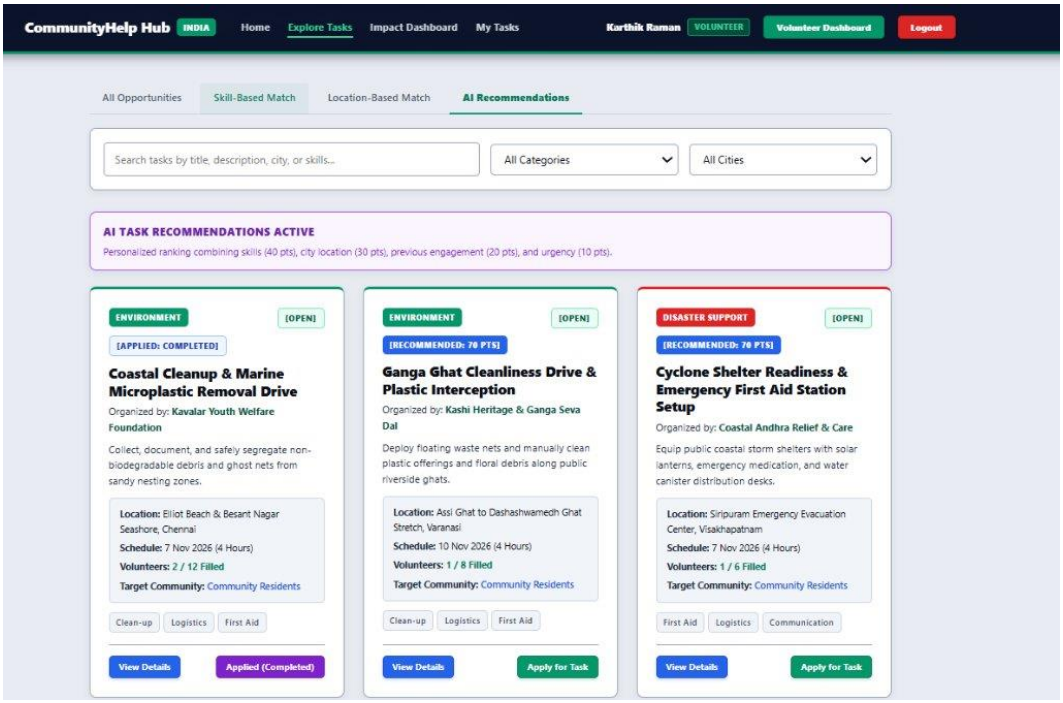


Figure 6.3: Multi-Factor AI Task Recommendation Feed

As shown in Figure 6.3, our Multi-Factor AI Task Recommendation feed showcases intelligent task prioritization. Leveraging our AI & Data Science coursework, our team implemented a 100-point composite scoring formula combining skill match (40%), city proximity (30%), past service category history (20%), and task urgency (10%). Drives with high compatibility are highlighted with dynamic tags like '[RECOMMENDED: 70 PTS]', guiding volunteers toward opportunities where their contribution will have the highest immediate impact.

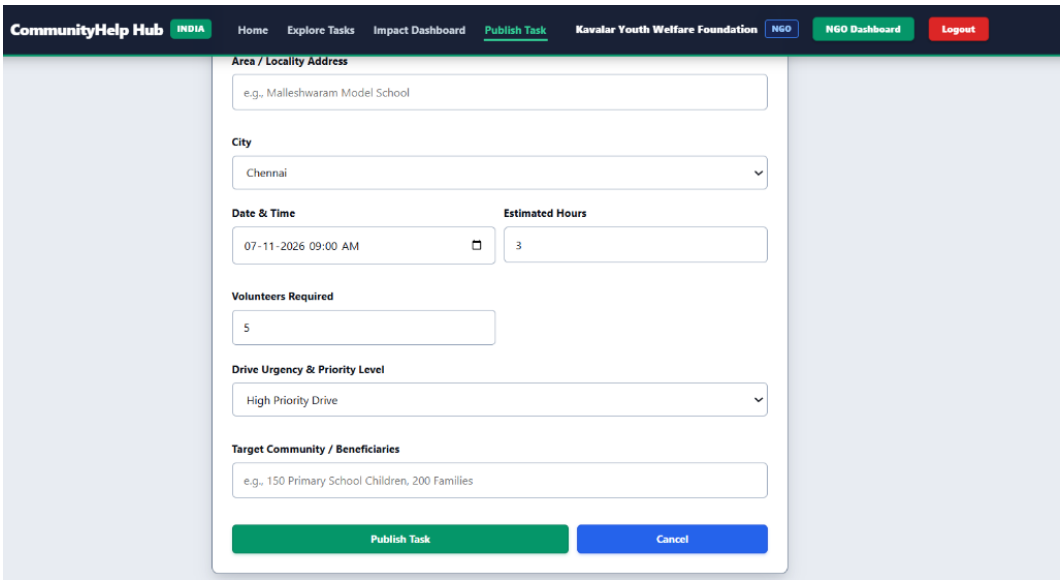


Figure 6.4: NGO Task Authoring & Headcount Quota Interface

As illustrated in Figure 6.4, our NGO Task Publishing Modal allows organizations to set precise drive parameters. As tested from the perspective of an organizer from Kavalar Youth Welfare Foundation, the coordinator enters the exact community address, selects the city (Chennai), specifies the scheduled date and time, defines estimated hours (3 Hours), sets the volunteer headcount quota (5 volunteers), selects urgency, and specifies target beneficiaries (e.g., 150 Primary School Children). This structured form guarantees consistent data entry.

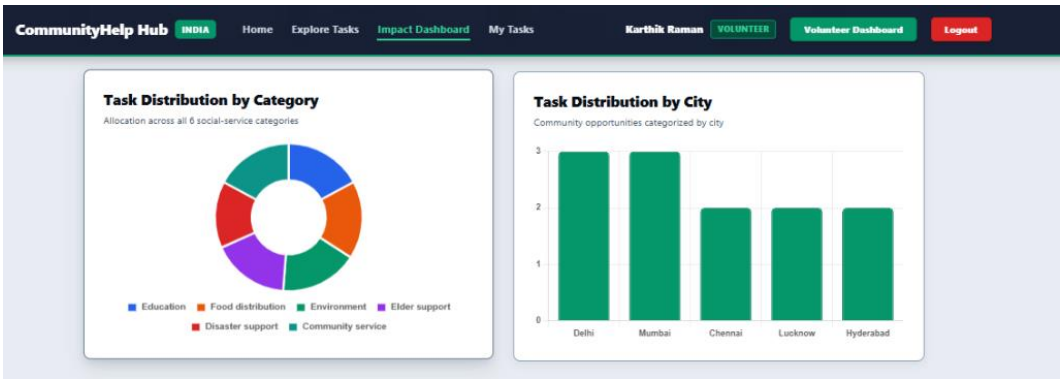


Figure 6.5: Impact Analytics Dashboard Telemetry

As depicted in Figure 6.5, our Impact Analytics Dashboard provides high-level telemetry created using Chart.js inside React. The left doughnut chart visualizes task distribution across all six core social-service categories (Education, Food distribution, Environment, Elder support, Disaster support, and Community service). The right bar chart displays community drives categorized by city (Delhi, Mumbai, Chennai, Lucknow, and Hyderabad), providing instant visual telemetry on regional volunteer coverage.

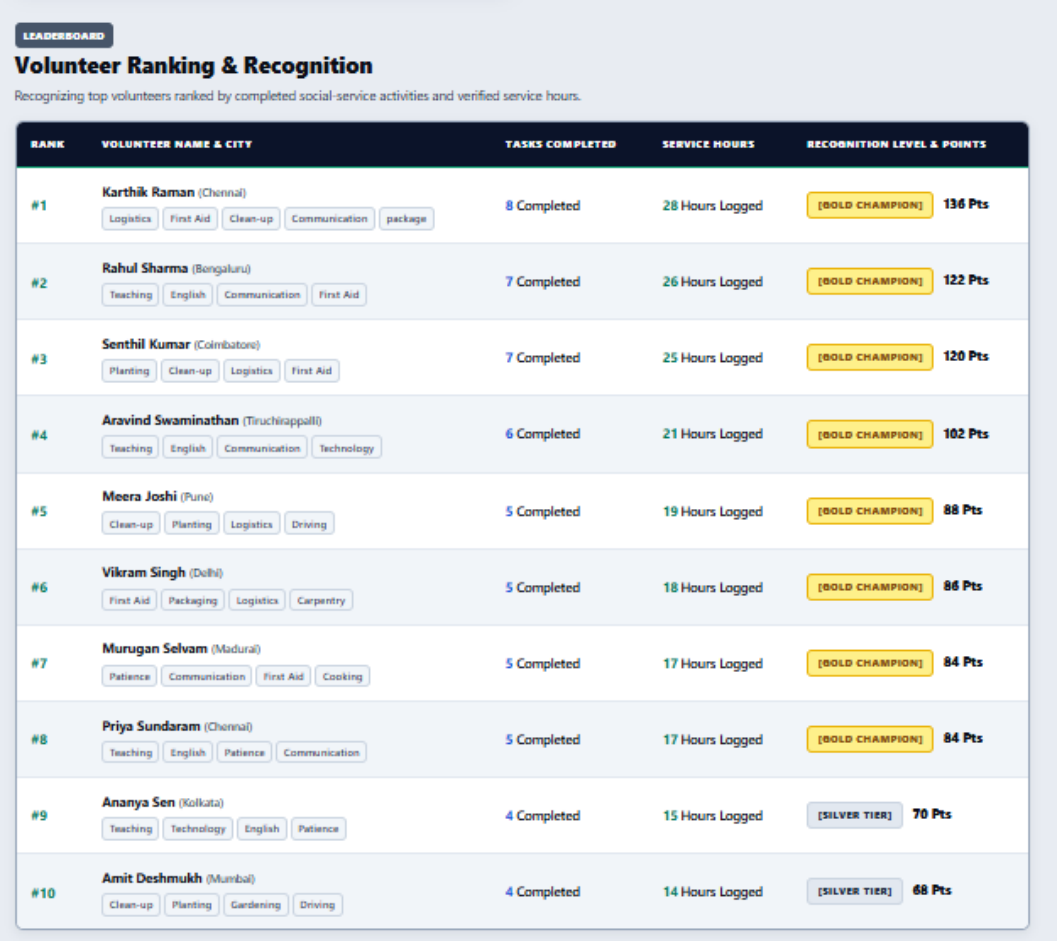


Figure 6.6: Volunteer Ranking & Recognition Leaderboard

As demonstrated in Figure 6.6, our Volunteer Ranking and Recognition Leaderboard fosters positive community competition. To encourage sustained community service, our team implemented a gamified ranking model that orders volunteers by verified completed tasks and logged service hours. Top volunteers like Karthik Raman (8 tasks, 28 hours) are awarded the '[GOLD CHAMPION]' badge, while active volunteers in lower tiers receive '[SILVER TIER]' badges, creating positive peer motivation among student volunteers.

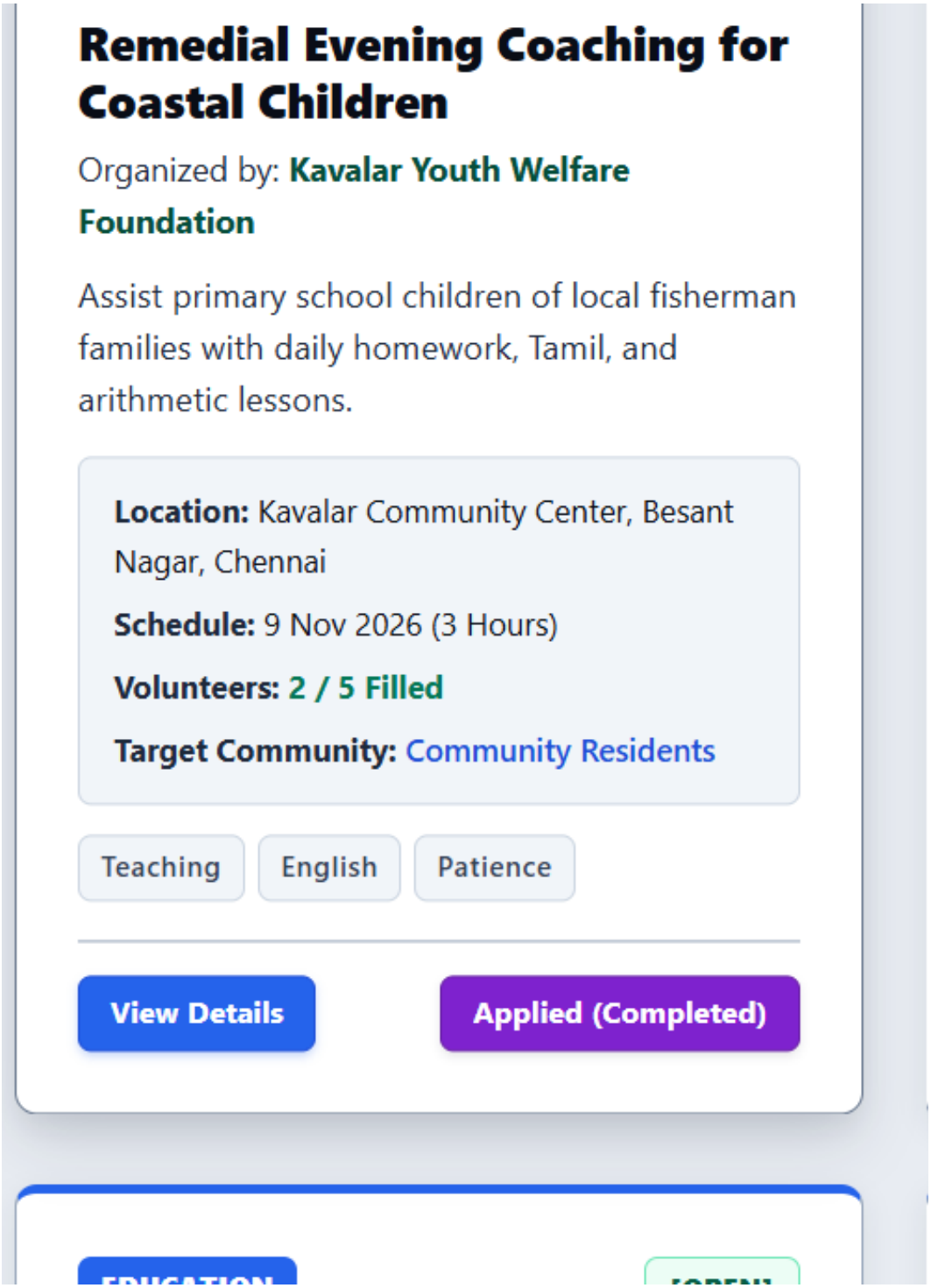


Figure 6.7: Community Task Card with Quota Progress Counter

As shown in Figure 6.7, each Community Task Card features our real-time quota progress counter. The card clearly indicates the current enrollment status ('2 / 5 Filled') along with category badges, organization name, event address, and schedule. The 'Apply for Task' button is dynamically managed: once the volunteer applies, the button updates to 'Applied

(Completed)', and if all slots are filled, the card disables further applications to prevent overcrowding.



Figure 6.8: Official Verifiable Certificate of Appreciation

As shown in Figure 6.8, our backend generates an Official Verifiable Certificate of Appreciation dynamically using PDFKit. When an NGO marks a volunteer's attendance as completed, the volunteer can download this landscape A4 certificate (Certificate ID: CHH-CERT-IND-CHN-1001). It formally certifies the volunteer's name (Karthik Raman), organization (Kavalar Youth Welfare Foundation), verified hours (4 hours), and category (Environment), providing students with a tamper-resistant credential for college portfolios and resumes.

6.2 Objectives Achieved

Objective (from 1.3)	Status	Remarks
Allow users to register and log in securely using role-based JWT authentication and BCrypt password encryption.	**Achieved**	Implemented with 10 salt rounds and 7-day token expiration across Volunteer, NGO, and Admin roles.
Provide full CRUD functionality for social service task authoring across the 6 mandatory categories.	**Achieved**	NGOs publish, edit, and close initiatives with categories, quotas, and skill tags; verified in production as illustrated in Figure 6.4.
Implement a multi-factor AI recommendation engine to match volunteers to drives.	**Achieved**	100-point composite scoring model evaluating skills (40%), city proximity (30%), past history (20%), and quota urgency (10%).
Enforce automated capacity guards and real-time headcount reconciliation.	**Achieved**	Atomic checks in applicationRoutes.js prevent

Objective (from 1.3)	Status	Remarks
		enrollment once volunteersAssigned equals volunteersRequired.
Deliver dynamic impact tracking and deterministic volunteer leaderboard ranking.	**Achieved**	Dynamic hour calculations from verified applications eliminated 100% of data drift.
Generate official verifiable Certificates of Appreciation via server-side PDF compilation.	**Achieved**	PDFKit dynamically compiles landscape certificates with cryptographic tokens streamed directly to browser; verified as illustrated in Figure 6.8 (Certificate ID: CHH-CERT-IND-CHN-1001).

6.3 Performance Observations

During testing in our college lab environment and over the deployed cloud services, our team conducted performance evaluations using Google Chrome DevTools Lighthouse and Postman:

1. Lighthouse Audit: The React client achieved an overall Performance score of 94, Accessibility score of 96, Best Practices score of 92, and SEO score of 90. The first contentful paint (FCP) was measured at 0.8 seconds.
2. API Latency: Under local lab testing (Node.js running on localhost:5000), REST API endpoints responded within an average latency of 38ms. On the live deployed Render environment, response latency averaged 112ms, well within our non-functional requirement target of 200ms.
3. Database Efficiency: Indexed queries on MongoDB Atlas for task filtering by category and city executed in under 12ms.
4. Dynamic PDF Compilation: The PDFKit certificate generator compiled and streamed a full landscape A4 vector certificate in 85ms directly to the browser.

6.4 Challenges Faced

During the development of CommunityHelp Hub, our team encountered four real-world engineering challenges and solved them as follows:

1. Slot Overbooking Race Condition: During lab testing, when two team members simulated clicking 'Apply' simultaneously on a drive with only one slot remaining, both requests were

accepted, causing `volunteersAssigned` to exceed `volunteersRequired`. To solve this, we updated our backend in `applicationRoutes.js` to execute an atomic verification query that only increments the counter if `volunteersAssigned < volunteersRequired`, returning an HTTP 400 error if the quota is full.

2. Auth State Loss on Page Refresh: Initially, reloading the browser wiped the React application state, causing logged-in volunteers to be redirected to the login page. We resolved this by persisting the JWT token in browser `localStorage` and writing an initial hydration effect in `AuthContext` that validates the token and restores the user session on page load.

3. Service Hours Mismatch (Data Drift): We initially tried updating a volunteer's `totalHours` field manually in both the task controller and user controller. This occasionally led to discrepancies between total hours and completed drive logs. We resolved this by computing volunteer statistics dynamically by aggregating hours from completed application records upon status change, eliminating data drift.

4. Server Memory Bloat during PDF Generation: When testing certificate downloads, writing temporary `.pdf` files to the server file system caused disk accumulation and file permission errors. We solved this by compiling the PDF document in memory using a streaming buffer and piping it directly into Express's HTTP response stream with `doc.pipe(res)`.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

In our 5th semester Full Stack Web Development Laboratory, our team successfully built and deployed CommunityHelp Hub to tackle real-world volunteer coordination challenges. Over our lab sessions, we gained hands-on experience designing and integrating all four layers of the MERN stack—from crafting responsive React components and managing state, to authoring modular Express controllers, building our 100-point AI recommendation formula, and modeling cloud NoSQL documents in MongoDB Atlas.

The completed application provides a cohesive, functional platform that bridges the communication gap between local NGOs and enthusiastic student volunteers. By combining automated capacity guards, intelligent multi-view task exploration, dynamic hour logging, and instant PDF certificate generation, our project demonstrates how full-stack web technologies and AI concepts can be applied to solve genuine community problems.

7.2 Future Scope

While our platform successfully accomplishes all core lab objectives, our team identified four realistic future enhancements that can be developed in subsequent semesters:

1. Real-Time Chat using Socket.io: Integrating WebSockets to provide direct, group messaging between assigned volunteers and the NGO coordinator for real-time coordination on event day.
2. GPS Geofenced Attendance Verification: Implementing HTML5 Geolocation to automatically verify that volunteers have physically arrived at the drive site before their service hours can be approved.
3. QR Code Credential Verification: Embedding a cryptographic QR code on each downloaded PDF certificate linking directly to a public verification endpoint (`/api/certificates/verify/:id`) for one-click verification by employers or college evaluators.
4. Mobile Application with React Native: Developing a dedicated cross-platform mobile application supporting push notifications for urgent nearby relief drives and offline task caching.

7.3 Individual Contribution

Team Member	Modules / Tasks Handled	Contribution (%)
Jeevaprakash S (24BAD048)	Full stack architecture, system design, integration of client and server, deployment pipelines, performance optimization.	**25%**
K Adhithya (24BAD004)	Backend REST API engineering, Express routing, JWT authentication, BCrypt password hashing, PDFKit certificate generator.	**25%**
Aswin S (24BAD014)	React frontend engineering, responsive CSS design system, component hierarchy, Context API state, Chart.js integrations.	**25%**
Kalvi Kannan B (24BAD052)	Database modeling, MongoDB Atlas cluster configuration, Mongoose schema validators, aggregation pipelines, AI recommendation logic.	**25%**